
CODES CORRECTEURS D'ERREURS

pour l'UE HAI906I "Calcul formel avancé et applications"

POLYCOPIÉ - VERSION DU 01/10/2026

ROCCO MORA

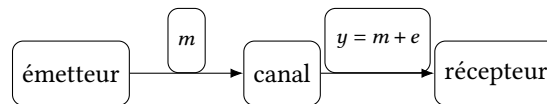
TABLE DES MATIÈRES

CHAPITRE 1	INTRODUCTION	PAGE 3
CHAPITRE 2	CODES (EN BLOCS) : NOTIONS DE BASE	PAGE 5
2.1	Alphabets, codes, distance, taux	5
2.2	Canaux de communication	8
2.3	Un code à chiffre de contrôle du monde réel : le numéro de sécurité sociale français	11
2.4	Premier aperçu de bons et de mauvais codes	12
2.5	Équivalence de codes	13
CHAPITRE 3	CODES LINÉAIRES	PAGE 14
3.1	Matrices génératrice et de contrôle de parité, dualité	14
3.2	Somme et intersection de codes	17
3.3	Poinçonnage et raccourcissement : fabriquer de nouveaux codes	18
3.4	Une galerie de familles classiques	20
3.5	Les codes non linéaires peuvent tout de même l'emporter	21

Introduction

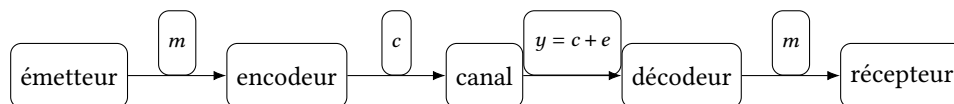
Le 20 août 1977, la sonde Voyager 2 quittait la Terre en emportant, parmi d'autres instruments, une caméra dont les images allaient devoir parcourir des milliards de kilomètres à travers le bruit interplanétaire avant d'atteindre une antenne radio au sol. Lorsque le signal arrivait, il était noyé dans un bruit thermique bien en deçà du niveau qu'une oreille ou un œil humain pouvait décoder directement, et pourtant les images de Jupiter, Saturne, Uranus et Neptune qui nous sont parvenues sont nettes. L'écart entre un canal bruité et un message fiable est comblé par les *codes correcteurs d'erreurs* : un filet de sécurité algébrique qui permet à un récepteur de reconstituer exactement ce qui a été envoyé, même si ce qui arrive réellement est corrompu.

Un exemple jouet. Avant tout formalisme, il est utile de voir l'idée centrale sur le plus petit exemple possible. Chaque fois que nous communiquons numériquement, nous le faisons à travers un *canal bruité* : ce que reçoit le récepteur n'est pas tout à fait ce qu'a envoyé l'émetteur.



Disons que l'émetteur souhaite transmettre le message binaire $m = (1, 0, 1)$, et que le canal inverse un bit, si bien que le récepteur reçoit $y = (0, 0, 1)$. En écrivant $y = m + e$, le *vecteur d'erreur* est ici $e = (1, 0, 0)$ — mais le récepteur ne voit jamais que y , jamais m ni e séparément, et n'a aucun moyen de savoir que quelque chose s'est mal passé : $(0, 0, 1)$ semble tout aussi plausible comme message que ne l'était $(1, 0, 1)$.

La solution de la théorie des codes n'est pas de modifier le canal — nous ne le pouvons pas — mais de modifier ce que nous envoyons. Nous *encodons* m au préalable, en ajoutant délibérément de la redondance, de sorte que les séquences transmises possibles (les *mots de code*) se trouvent loin les unes des autres.



Par exemple, au lieu d'envoyer directement $m = (1, 0, 1)$, répétons-le trois fois : $c = (1, 0, 1, 1, 0, 1, 1, 0, 1)$. Si la même erreur d'un seul bit se produit, le récepteur reçoit alors $y = (1, 0, 0, 1, 0, 1, 1, 0, 1)$ — et comme les huit mots de code possibles (un pour chaque choix de $m \in \{0, 1\}^3$) sont largement espacés les uns des autres, il existe exactement *un seul* mot de code à un bit inversé de y , à savoir c lui-même : l'erreur n'est pas seulement détectée mais *corrigée*, et m est retrouvé exactement, sans aucune ambiguïté.

Cet exemple jouet contient déjà tous les ingrédients auxquels le reste de ce cours consacrera de nombreux chapitres pour les préciser et les rendre efficaces : une notion de *distance* entre séquences, pour que « éloignés » et « mot de code le plus proche » aient un sens précis (Section 2.1); un *encodeur* transformant des messages courts en mots de code plus longs et bien séparés, et ce de façon systématique plutôt que par répétition ad hoc (Chapitre 3); et un *décodeur* qui, étant donné y , doit trouver efficacement le mot de

code le plus proche — trivial par force brute pour des messages de 3 bits, totalement infaisable par force brute dès que m compte un millier de bits (Chapitre 5) — le tout soumis à des compromis difficiles entre la quantité de redondance que nous pouvons nous permettre d'ajouter et le nombre d'erreurs que nous pouvons nous permettre de corriger (Chapitre 4).

La même idée, sous des habits différents, se retrouve dans chaque disque compact, chaque paquet Wi-Fi, chaque QR code sur une tasse de café, chaque baie RAID et chaque contrôleur SSD, et — plus surprenant encore — dans les schémas cryptographiques conçus pour résister aux ordinateurs quantiques, ainsi que dans les protocoles de partage de secret qui permettent d'ouvrir le coffre d'une banque seulement lorsque trois directeurs sur cinq sont d'accord.

Deux racines historiques. Le sujet trouve véritablement son origine dans le même bâtiment, aux Bell Telephone Laboratories, vers 1948. **Claude Shannon**, dans son article *A Mathematical Theory of Communication* (1948), montre, de façon purement existentielle et probabiliste, qu'un canal possède une *capacité* en deçà de laquelle une communication arbitrairement fiable est possible, et au-delà de laquelle elle ne l'est pas — mais l'argument de Shannon n'exhibe aucun code utilisable. **Richard Hamming**, qui travaillait dans le couloir voisin sur des calculateurs à relais s'arrêtant régulièrement à cause d'erreurs de parité détectées par des lecteurs de cartes perforées, posa la question complémentaire et constructive : étant donné qu'une erreur a été *détectée*, peut-elle aussi être *localisée et corrigée*? Son article de 1950, *Error Detecting and Error Correcting Codes*, répond par l'affirmative et donne la première famille non triviale de codes, encore utilisée aujourd'hui. Un troisième nom couramment associé à cette période initiale est celui de **Marcel Golay**, qui publia en 1949 (dans un article d'à peine une page) deux codes exceptionnels qui demeurent, soixante-quinze ans plus tard, parmi les objets combinatoires les plus remarquables connus.

Point de vue de ce cours. La théorie de Shannon porte sur la capacité *probabiliste*; la théorie que nous développons ici est *combinatoire* : au lieu des capacités de canal et des probabilités d'erreur, notre objet central est la **distance de Hamming** entre deux chaînes, et notre question centrale est de savoir combien de mots de code, deux à deux éloignés selon cette distance, peuvent être placés dans un espace fini. C'est ce point de vue combinatoire qui rend possibles des constructions explicites et des algorithmes de décodage explicites.

Jouons au jeu du théoricien des codes. Avant d'étudier rigoureusement la théorie des codes correcteurs d'erreurs, ses problèmes fondamentaux et les solutions trouvées au fil des décennies, il peut être intéressant d'essayer de construire nous-mêmes quelques codes. Par exemple :

1. Construire un code capable de toujours détecter deux erreurs.
2. Construire un code capable de toujours corriger deux erreurs.
3. Construire un code capable de toujours détecter une erreur en n'ajoutant qu'un seul bit de redondance.
4. Construire un code capable de toujours corriger une erreur en n'ajoutant qu'un seul bit de redondance.

Certaines de ces questions sont-elles plus faciles que d'autres ? Pensez-vous qu'elles sont toutes réalisables, ou que certaines sont théoriquement impossibles ?

Codes (en blocs) : notions de base

§ 2.1 Alphabets, codes, distance, taux

Definition 2.1. Soit A un ensemble fini (un *alphabet*) avec $|A| = q$, et soit $n \in \mathbb{N}^*$. Un *code de longueur n sur A* est simplement un sous-ensemble $C \subseteq A^n$. Ses éléments sont des *mots de code*. On note $M = |C|$ sa taille.

Aucune structure algébrique sur A ou C n'est supposée à ce stade — c'est délibéré : certains des meilleurs codes connus pour de petits paramètres ne sont pas linéaires, et il est important de voir le cadre général avant de se spécialiser.

Definition 2.2. Le *poids de Hamming* de $x \in A^n$ est

$$\text{wt}(x) := |\{i \in \{1, \dots, n\} \mid x_i \neq 0\}|.$$

Definition 2.3. La *distance de Hamming* entre $x, y \in A^n$ est

$$d(x, y) := |\{i \in \{1, \dots, n\} \mid x_i \neq y_i\}|.$$

On peut voir de façon équivalente la distance de Hamming entre $x, y \in A^n$ comme $\text{wt}(x - y)$.

Proposition 2.4. d est une *métrique* sur A^n , c'est-à-dire que

1. elle est *positive*, $d(x, y) \geq 0$ et $d(x, y) = 0$ si et seulement si $x = y$,
2. elle est *symétrique*, $d(x, y) = d(y, x)$,
3. et elle satisfait l'*inégalité triangulaire* $d(x, z) \leq d(x, y) + d(y, z)$.

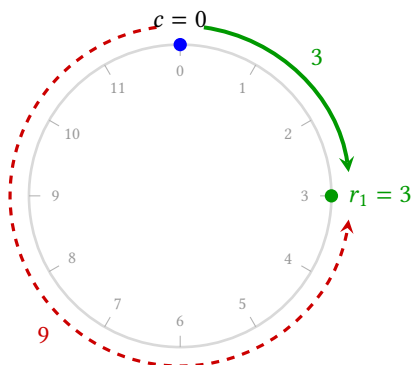
Exercice 2.5. Démontrer la Proposition 2.4.

Remark 2.6 (D'autres métriques existent). La distance de Hamming est la métrique que ce cours utilise presque exclusivement, mais ce n'est pas la seule qui présente un intérêt. Lorsque l'alphabet lui-même porte une structure cyclique, en « cadran d'horloge » — par exemple $A = \mathbb{Z}_q = \{0, 1, \dots, q-1\}$, par opposition à un simple ensemble fini non structuré — la *distance de Lee*

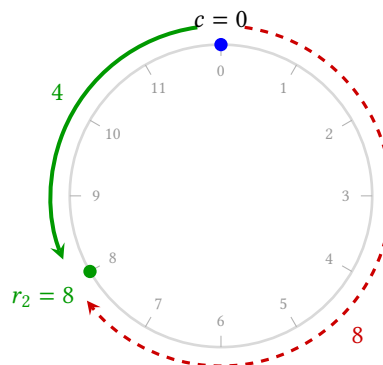
$$d_L(x, y) := \sum_{i=1}^n \min(|x_i - y_i|, q - |x_i - y_i|)$$

est souvent celle qui a un sens physique : elle mesure l'écart de chaque coordonnée comme le plus court

chemin autour d'un cycle de longueur q , plutôt que comme un simple indicateur de correspondance/non-correspondance, ce qui modélise les erreurs de type « phase » (par exemple dans certains schémas de modulation numérique) bien plus fidèlement que ne le fait la distance de Hamming. Voir l'Exercice 2.7 ci-dessous pour une preuve que d_L est bien une métrique, et pour sa relation avec d .

Exemple 1 dans $\mathbb{Z}_{12}$ 

$$d_L(0, 3) = \min(3, 9) = 3$$

Exemple 2 dans $\mathbb{Z}_{12}$ 

$$d_L(0, 8) = \min(8, 4) = 4$$

Un autre exemple est donné par la métrique de rang : dans ce cas, les mots de code ne sont pas des vecteurs mais des matrices, le poids est calculé comme le rang de la matrice et la distance comme le rang de la différence.

Exercice 2.7 (La métrique de Lee). Soit $A = \mathbb{Z}_q$ et rappelons la distance de Lee $d_L(x, y) := \sum_{i=1}^n \min(|x_i - y_i|, q - |x_i - y_i|)$ de la Remarque 2.6.

- Montrer que d_L est une métrique sur $\mathbb{Z}_q^n$: positivité, symétrie, $d_L(x, y) = 0 \iff x = y$, et l'inégalité triangulaire. (Pour l'inégalité triangulaire, commencer par prouver le cas coordonnée par coordonnée, $n = 1$; puis sommer sur les coordonnées.)
- Montrer que $d_L(x, y) \geq d(x, y)$ toujours, avec égalité lorsque $q \in \{2, 3\}$, et donner un exemple explicite avec $q \geq 4$ où l'inégalité est stricte.

Définition 2.8. La *distance minimale* d'un code C ayant au moins deux mots de code est

$$d(C) := \min_{\substack{x, y \in C \\ x \neq y}} d(x, y).$$

On note les paramètres de C par $(n, M, d)_q$, ou simplement (n, M, d) lorsque q est clair. Le *taux* (d'information) de C est $R := \log_q(M)/n \in [0, 1]$, et sa *distance relative* est $\delta := d/n \in [0, 1]$.

Exercice 2.9 (L'inclusion contrôle la distance). Soient $C_1 \subseteq C_2 \subseteq A^n$ deux codes, chacun ayant au moins deux mots de code.

- Montrer que $d(C_1) \geq d(C_2)$.
- Donner un exemple explicite, avec $C_1 \subsetneq C_2 \subseteq \mathbb{F}_2^4$ de tailles toutes deux ≥ 2 , où l'inégalité est stricte ($d(C_1) > d(C_2)$), ainsi qu'un autre exemple où il y a égalité.

Il existe une tension entre R et δ : intuitivement, écarter M mots de code de sorte qu'ils soient deux à deux très éloignés (grand δ) laisse moins de « place » pour en placer beaucoup (grand R). Cette tension est le mieux capturée par une unique quantité combinatoire, qu'il vaut la peine d'isoler dès maintenant puisqu'elle organisera pour ainsi dire tout ce qui suit.

Définition 2.10 (Le problème combinatoire central). Pour des entiers $n, d \geq 1$ et $q \geq 2$, on définit

$$A_q(n, d) := \max\{|C| \mid C \subseteq A^n, |A| = q, d(C) \geq d\},$$

la plus grande taille possible pour *n'importe quel* code (linéaire ou non) de longueur n sur un alphabet de taille q garantissant une distance minimale au moins d . Un code $C \subseteq A^n$ avec $d(C) \geq d$ atteignant $|C| = A_q(n, d)$ est dit $A_q(n, d)$ -*optimal* (ou simplement *optimal* pour ces paramètres, lorsque q, n, d sont clairs d'après le contexte).

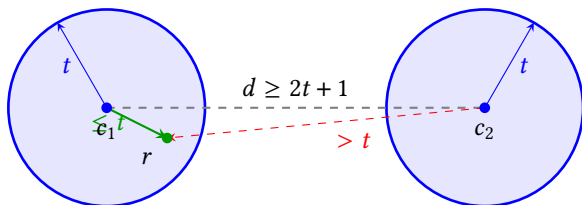
Déterminer $A_q(n, d)$ exactement, pour tous n, d, q , est sans doute *le* problème combinatoire central de tout ce cours : c'est précisément la question « à quel point un code peut-il être bon ? », dépouillée de toute considération hormis la longueur, la taille de l'alphabet et la distance minimale garantie. Le Chapitre 4 développe plusieurs bornes générales sur $A_q(n, d)$ (Singleton, Hamming, Plotkin, Gilbert–Varshamov, Griesmer), chacune capturant une obstruction combinatoire ou un argument d'existence différent ; pour ainsi dire chaque famille spécifique de codes construite plus loin dans ce cours est, au fond, une tentative de cerner exactement $A_q(n, d)$ pour une certaine famille infinie de paramètres — ou de s'en approcher utilement avec une construction qui se trouve aussi être efficacement *encodable et décodable*, ce dont $A_q(n, d)$ seul ne dit rien. Même pour $q = 2$ et des n assez modestes, la valeur exacte de $A_2(n, d)$ reste inconnue pour une infinité de paires (n, d) : après soixante-quinze ans, il s'agit toujours d'un domaine ouvert de la combinatoire énumérative, dont les records actuels sont répertoriés en ligne (<https://www.codetables.de/>).

Exercice 2.11. Soit C un code $A_2(n, d)$ -optimal (Définition 2.10) avec d impair. Montrer qu'ajouter un bit de parité global à chaque mot de code (*extension*) transforme C en un code de longueur $n + 1$ et de distance minimale $d + 1$, de même taille. En déduire que $A_2(n + 1, d + 1) \geq A_2(n, d)$ pour tout d impair.

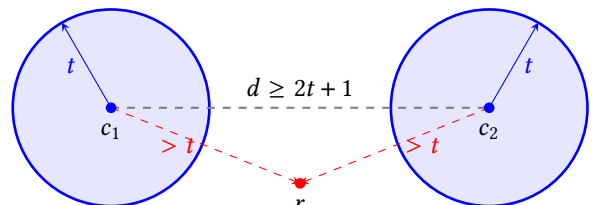
Proposition 2.12 (Distance et correction d'erreurs). Si $d(C) \geq 2t + 1$, alors pour tout $c \in C$ et tout $y \in A^n$ avec $d(y, c) \leq t$, le mot de code c est l'unique élément de C à distance au plus t de y . Par conséquent, un code de distance minimale d peut corriger tout motif de $t = \lfloor (d - 1)/2 \rfloor$ erreurs, et peut détecter (sans nécessairement les corriger) tout motif d'au plus $d - 1$ erreurs.

Démonstration. Si $c' \in C$, $c' \neq c$, vérifiait aussi $d(y, c') \leq t$, l'inégalité triangulaire donnerait $d(c, c') \leq 2t < d(C)$, ce qui contredit la définition de $d(C)$. ■

1 solution



0 solutions



Exercice 2.13. Soit $A = \{0, 1\}$ ($q = 2$) et considérons le code $C = \{(1, 1, 0, 1, 0), (0, 0, 0, 1, 1), (0, 1, 1, 0, 1)\} \subseteq A^5$.

- Calculer les trois poids de Hamming et les trois distances de Hamming deux à deux entre les trois mots de code.
- En déduire $d(C)$, et indiquer le plus grand t pour lequel C est garanti de détecter/corriger t erreurs quelconques.

(c) $y = (0, 1, 1, 1, 0)$ est-il à distance t d'un unique mot de code ? Si oui, lequel ?

Exercice 2.14. Soit $A = \{0, 1, 2\}$ ($q = 3$) et considérons le code $C = \{(0, 0, 1, 2), (1, 2, 0, 0), (2, 1, 2, 1), (0, 1, 1, 0)\} \subseteq A^4$.

- (a) Calculer les quatre poids de Hamming et les six distances de Hamming deux à deux entre les quatre mots de code.
- (b) En déduire $d(C)$, et indiquer le plus grand t pour lequel C est garanti de détecter/corriger t erreurs quelconques.
- (c) $y = (0, 1, 2, 0)$ est-il à distance t d'un unique mot de code ? Si oui, lequel ?

Définition 2.15. La *boule* de rayon t autour de $x \in A^n$ est $B(x, t) := \{z \in A^n \mid d(x, z) \leq t\}$, la *sphère* de rayon t autour de $x \in A^n$ est $S(x, t) := \{z \in A^n \mid d(x, z) = t\}$.

La Proposition 2.12 dit exactement que les boules $B(c, \lfloor (d-1)/2 \rfloor)$, $c \in C$, sont deux à deux disjointes — une image d'*empilement de sphères* qui réapparaîtra plus loin sous le nom de borne de Hamming.

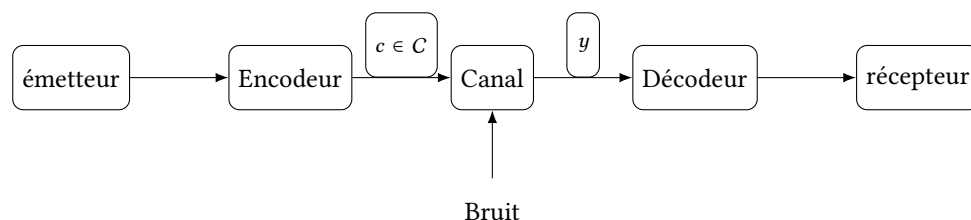
Proposition 2.16 (Cardinaux des boules et des sphères). Pour tout $x \in A^n$ et $0 \leq t \leq n$, $|S(x, t)| = \binom{n}{t}(q-1)^t$ et $|B(x, t)| = \sum_{i=0}^t \binom{n}{i}(q-1)^i$.

Exercice 2.17. Démontrer la Proposition 2.16.

Exercice 2.18. Sur $A = \{0, 1\}$, $n = 6$, calculer $|B(x, 2)|$ (le nombre de chaînes binaires de longueur 6 à distance de Hamming au plus 2 d'une chaîne fixée x). Recommencer pour $q = 4$, rayon $t = 3$, en utilisant la formule générale pour la taille des boules. Comparer chaque réponse à q^n et commenter la rapidité avec laquelle les boules remplissent l'espace ambiant lorsque t croît.

§ 2.2 Canaux de communication

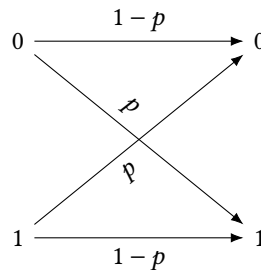
D'où viennent physiquement l'espace ambiant A^n et toute l'idée de corriger jusqu'à t erreurs ? Cette section précise, et généralise, l'image jouet du Chapitre 1 : là, un message m devenait un mot de code c , le canal ajoutait une erreur inconnue e , et le récepteur ne voyait que $y = c + e$. Nous fixons maintenant les modèles de bruit spécifiques auxquels ce cours se réfère chaque fois que nous demandons combien d'erreurs — ou combien de symboles *effacés* — un décodeur doit être construit pour tolérer.



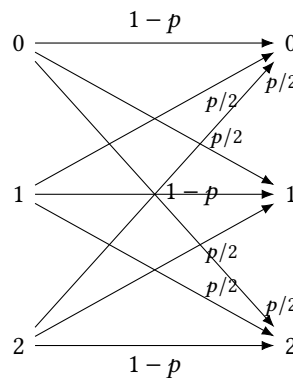
Définition 2.19. Un *canal discret* d'alphabet d'entrée A et d'alphabet de sortie B est spécifié par des probabilités de transition $P(y \mid x)$, $x \in A$, $y \in B$, avec $\sum_{y \in B} P(y \mid x) = 1$ pour tout x . Le canal est dit *sans mémoire* si, sur une séquence d'entrée $x = (x_1, \dots, x_n) \in A^n$, chaque symbole de sortie est produit indépendamment : $\Pr[y \mid x] = \prod_{i=1}^n P(y_i \mid x_i)$.

Tout canal utilisé dans ce cours est un canal discret sans mémoire (CDSM, ou DMC en anglais). Les canaux réels sont rarement *exactement* sans mémoire, mais le modèle sans mémoire est la bonne première approximation, et c'est celui par rapport auquel le théorème de capacité de Shannon et la plupart des constructions de ce cours sont énoncés.

Definition 2.20 (Canal binaire symétrique). Le *canal binaire symétrique* $BSC(p)$, $0 \leq p \leq \frac{1}{2}$, a $A = B = \{0, 1\}$ et inverse le bit transmis indépendamment avec probabilité p : $P(1 | 0) = P(0 | 1) = p$, $P(0 | 0) = P(1 | 1) = 1 - p$.



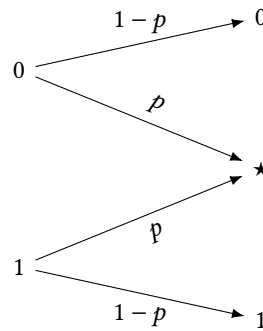
Definition 2.21 (Canal symétrique q -aire). Le *canal symétrique q -aire* $QSC_q(p)$ a $A = B = \{0, \dots, q-1\}$, transmet chaque symbole correctement avec probabilité $1 - p$, et sinon le remplace par *chacun* des $q-1$ autres symboles avec probabilité égale $p/(q-1)$: $P(x | x) = 1 - p$, $P(y | x) = p/(q-1)$ pour $y \neq x$.



(Représenté pour $q = 3$, où chaque flèche hors diagonale porte la probabilité $p/(q-1) = p/2$; en général, chaque entrée possède $q-1$ flèches hors diagonale, chacune de probabilité $p/(q-1)$.)

Les erreurs ne sont pas le seul type de corruption qu'il vaille la peine de modéliser séparément : parfois le récepteur sait exactement *quels* symboles ont été perdus (un secteur de disque qui n'a pas pu être lu, un paquet réseau qui n'est jamais arrivé) sans savoir ce qu'ils disaient — une situation qualitativement plus facile à gérer qu'une erreur en position inconnue, et suffisamment fréquente pour mériter son propre modèle de canal.

Definition 2.22 (Canal binaire à effacement). Le *canal binaire à effacement* $BEC(p)$ a pour alphabet d'entrée $\{0, 1\}$ et pour alphabet de sortie $\{0, 1, \star\}$, où $\star$ (l'*effacement*) signifie « perdu, position connue » : chaque bit est transmis correctement avec probabilité $1 - p$ et effacé avec probabilité p — il n'est *jamais* inversé vers la mauvaise valeur.



Definition 2.23 (Canal q -aire à effacement). Le canal q -aire à effacement généralise ceci à l'alphabet d'entrée $\{0, \dots, q-1\}$ et à l'alphabet de sortie $\{0, \dots, q-1, \star\}$: chaque symbole est transmis correctement avec probabilité $1-p$ et effacé avec probabilité p .

Exercice 2.24. Représenter graphiquement le canal q -aire à effacement.

Enfin, nous précisons la règle de décodage que ce cours a déjà utilisée de façon informelle, et que nous étudierons algorithmiquement dans toute sa généralité plus avant.

Definition 2.25 (Décodage au plus proche voisin). Étant donné un mot reçu $y \in B^n$, le décodage au plus proche voisin produit

$$\hat{c}_{\text{NN}} := \arg \min_{c \in C} d(y, c)$$

(en départageant arbitrairement les ex æquo), c'est-à-dire un mot de code le plus proche de y pour la distance de Hamming. Il s'agit d'une règle de décision purement géométrique indépendante des caractéristiques probabilistes du canal.

Definition 2.26 (Décodage au maximum de vraisemblance). Étant donné un mot reçu $y \in B^n$, le décodage au maximum de vraisemblance (ML, pour *Maximum Likelihood*) produit

$$\hat{c}_{\text{ML}} := \arg \max_{c \in C} \Pr[y | c]$$

(en départageant arbitrairement les ex æquo), c'est-à-dire un mot de code qui maximise la probabilité d'observer la séquence y sachant que c a été transmis.

Exercice 2.27 (Rapprochement des décodages sur un canal sans mémoire $\text{QSC}_3(p)$). Soit un canal symétrique 3-aire $\text{QSC}_3(p)$ de paramètre d'erreur $p < 2/3$. Considérons un code $C \subseteq \{0, 1, 2\}^n$ et un vecteur reçu $y \in \{0, 1, 2\}^n$.

- Expression de la vraisemblance : Pour un mot de code $c \in C$ situé à distance de Hamming $d = d(y, c)$ de y , exprimer la probabilité de transition $\Pr[y | c]$ en fonction de n , d et p .
- Équivalence des règles : Montrer que la condition $p < 2/3$ implique que la fonction $d \mapsto \Pr[y | c]$ est strictement décroissante en fonction de d . En déduire que les règles de décodage au plus proche voisin ($\hat{c}_{\text{NN}}$) et au maximum de vraisemblance ($\hat{c}_{\text{ML}}$) mènent aux mêmes choix de mots de code.

Exercice 2.28 (Divergence des règles de décodage sur un canal avec mémoire). Considérons l'alphabet binaire $A = \{0, 1\}^3$ de longueur $n = 3$ et le code $C = \{(0, 0, 0), (1, 1, 1)\}$. Supposons que le

canal ne soit pas sans mémoire et génère le mot reçu $y = c + e \pmod{2}$ selon la loi de probabilité suivante pour le motif d'erreur $e \in \{0, 1\}^3$:

$$\Pr[e] = \begin{cases} 0,80 & \text{si } e = (0, 0, 0), \\ 0,15 & \text{si } e = (0, 1, 1), \\ 0,01 & \text{pour chacun des 6 autres motifs d'erreur d'un poids } \text{wt}(e) \in \{1, 2, 3\}. \end{cases}$$

Supposons que le mot reçu soit $y = (0, 1, 1)$.

- Calculer la distance de Hamming entre y et chacun des deux mots de code de C . Quel mot de code est choisi par le décodeur au plus proche voisin ($\hat{c}_{\text{NN}}$) ?
- Calculer la probabilité conditionnelle $\Pr[y \mid c]$ pour chacun des deux mots de code $c \in C$. Quel mot de code est choisi par le décodeur au maximum de vraisemblance ($\hat{c}_{\text{ML}}$) ?

§ 2.3 Un code à chiffre de contrôle du monde réel : le numéro de sécurité sociale français

Tous les schémas de contrôle d'erreur utilisés au quotidien ne sont pas des constructions algébriques sophistiquées ; certains des plus courants ne sont qu'une simple vérification modulaire, et il vaut la peine de les voir explicitement avant que la machinerie des chapitres suivants ne fasse paraître tout plus compliqué qu'il ne l'est réellement.

Le *numéro d'inscription au répertoire* (NIR, couramment appelé « numéro de sécurité sociale ») est un nombre N à 13 chiffres codant le sexe, l'année et le mois de naissance, un code de département (et, pour les personnes nées à l'étranger, un code pays), la commune de naissance, et un numéro d'ordre. Pour se prémunir contre les erreurs de transcription, une *clé de contrôle* $K \in \{01, \dots, 97\}$ à 2 chiffres est ajoutée, calculée comme

$$K := 97 - (N \bmod 97)$$

(avec la convention qu'un code de département « 2A » ou « 2B », pour les personnes nées en Corse, est remplacé respectivement par « 19 » ou « 18 » avant de calculer $N \bmod 97$, et que $K = 97$ plutôt que $K = 0$ lorsque $97 \mid N$). La séquence complète à 15 chiffres (N, K) est ce qui apparaît effectivement sur une carte de sécurité sociale et une fiche de paie françaises.

Exemple 2.29.

$$\underbrace{2 \ 69 \ 05 \ 34 \ 172 \ 157}_{\text{NIR}} \quad \underbrace{64}_{\text{clé de contrôle}}$$

- 2 → sexe féminin (1 pour les hommes)
- 69 → année de naissance (1969)
- 05 → mois de naissance (mai)
- 34 → département de naissance (Hérault)
- 172 → code de la commune (Montpellier)
- 157 → 157-ème personne née cette année-là et ce mois-là dans cette commune (si > 999, un peu plus compliqué)
- 64 → clé de contrôle

Algébriquement, il s'agit d'une unique *équation de contrôle de parité sur $\mathbb{Z}_{97}$* : en notant $\bar{N} := N \bmod 97$, les paires valides sont exactement celles vérifiant $\bar{N} + K \equiv 0 \pmod{97}$ — l'analogue direct, sur le module 97 au lieu de $\mathbb{F}_2$, du code de contrôle de parité binaire de la section suivante. Le module 97 n'est pas arbitraire : c'est le plus grand nombre premier inférieur à 100 (de sorte que K tienne encore sur deux

chiffres), et c'est précisément la primalité qui rend les garanties de détection d'erreur du schéma (voir les exercices) démontrables plutôt qu'accidentelles.

Exercice 2.30. Vérifier la clé de contrôle avec votre propre numéro de sécurité sociale français.

Exercice 2.31 (Le NIR détecte toute erreur portant sur un seul chiffre). Supposons que le nombre véritable soit N avec la clé de contrôle $K = 97 - (N \bmod 97)$, et qu'exactement un chiffre de N soit mal transcrit (à une position $i \in \{0, \dots, 12\}$, en comptant à partir du chiffre des unités, le chiffre passant de d à $d' \neq d$), donnant un nombre corrompu $N' \neq N$, tandis que K lui-même est transmis correctement.

- (a) Montrer que $N' - N = (d' - d) \cdot 10^i$ pour un certain entier $d' - d$ avec $1 \leq |d' - d| \leq 9$.
- (b) En utilisant le fait que 97 est premier et ne divise pas 10 (donc $10^i \not\equiv 0 \pmod{97}$), montrer que $N' - N \not\equiv 0 \pmod{97}$.
- (c) En déduire que recalculer la clé de contrôle à partir de N' donne $97 - (N' \bmod 97) \neq K$: une erreur portant sur un seul chiffre est *toujours* détectée.

Exercice 2.32 (Deux erreurs peuvent s'annuler). Supposons maintenant que *deux* chiffres de N soient simultanément mal transcrits, aux positions $i \neq j$, changeant respectivement de $\Delta_i, \Delta_j \in \{-9, \dots, -1, 1, \dots, 9\}$ (donc $N' - N = \Delta_i 10^i + \Delta_j 10^j$).

- (a) Exhiber un choix explicite de i, j, Δ_i, Δ_j pour lequel $N' - N \equiv 0 \pmod{97}$ — c'est-à-dire que l'erreur passe complètement inaperçue. (Indication : fixer Δ_j et chercher parmi les 9 valeurs possibles de Δ_i celle, s'il en existe une, qui pour les i, j choisis rend la somme $\equiv 0 \pmod{97}$; ajuster i, j si la première tentative n'a pas de solution dans la plage autorisée.)
- (b) Supposons les positions $i \neq j$ fixées et Δ_i, Δ_j indépendants et uniformes sur les 9 différences de chiffres non nulles $\{-9, \dots, -1, 1, \dots, 9\}$. Montrer que pour chacune des 9 valeurs de Δ_j , *au plus* une valeur de Δ_i parmi les 9 possibles rend l'erreur indétectable (utiliser que deux éléments distincts de $\{-9, \dots, -1, 1, \dots, 9\}$ diffèrent de moins de 97, donc ne peuvent être congrus modulo 97 que s'ils sont égaux). En déduire que la probabilité qu'une double erreur soit indétectable est au plus $9/81 = 1/9$.

§ 2.4 Premier aperçu de bons et de mauvais codes

Exemple 2.33 (Répétition et parité). Sur $A = \{0, 1\}$, le *code de répétition* de longueur n , $\{0^n, 1^n\}$, a pour paramètres $(n, 2, n)_2$: excellente distance, taux exécrable. Le *code de contrôle de parité* $\{u \in \mathbb{F}_2^n \mid u_1 + \dots + u_n = 0\}$ a pour paramètres $(n, 2^{n-1}, 2)_2$: excellent taux, distance minimale — il ne fait que *détecter* une seule erreur, il ne peut pas la corriger. La majeure partie de ce cours consiste à trouver des compromis raisonnables entre ces deux extrêmes.

Exemple 2.34 (Un compromis parfait : le code de Hamming). Pour $r \geq 2$, posons $n = 2^r - 1$. Il existe un code binaire de longueur n , à 2^{n-r} mots de code, et de distance minimale 3 : il corrige une seule erreur tout en ne sacrifiant que $r = \log_2(n + 1)$ symboles de redondance — un taux tendant vers 1 lorsque n croît.

§ 2.5 Équivalence de codes

Renommer l'alphabet coordonnée par coordonnée, ou permuter les n positions, produit un code de paramètres identiques et de comportement de correction d'erreurs identique ; il est donc naturel d'identifier de tels codes.

Définition 2.35. Deux codes $C, C' \subseteq A^n$ sont *équivalents* si $C' = \{(\sigma_1(c_{\pi(1)}), \dots, \sigma_n(c_{\pi(n)})) \mid c \in C\}$ pour une certaine permutation $\pi \in S_n$ des coordonnées et certaines permutations $\sigma_1, \dots, \sigma_n$ de l'alphabet appliquées coordonnée par coordonnée.

Des codes équivalents ont la même taille et la même distribution des distances. La réciproque échoue de façon spectaculaire — des codes inéquivalents peuvent partager tous les mêmes paramètres (n, M, d) — un fait à garder à l'esprit chaque fois qu'un tableau (comme celui de la Section 3.5) ne liste que des paramètres et non les codes eux-mêmes.

Exercice 2.36. Démontrer que l'équivalence de codes (Définition 2.35) est une relation d'équivalence, et que des codes équivalents ont la même taille et la même distribution des distances $(A_0, A_1, \dots, A_n)$, où $A_i := |\{(x, y) \in C^2 : d(x, y) = i\}|/|C|$. Trouver un exemple de deux codes *inéquivalents* partageant les mêmes paramètres (n, M, d) .